PYTHON OOP TP REPORT

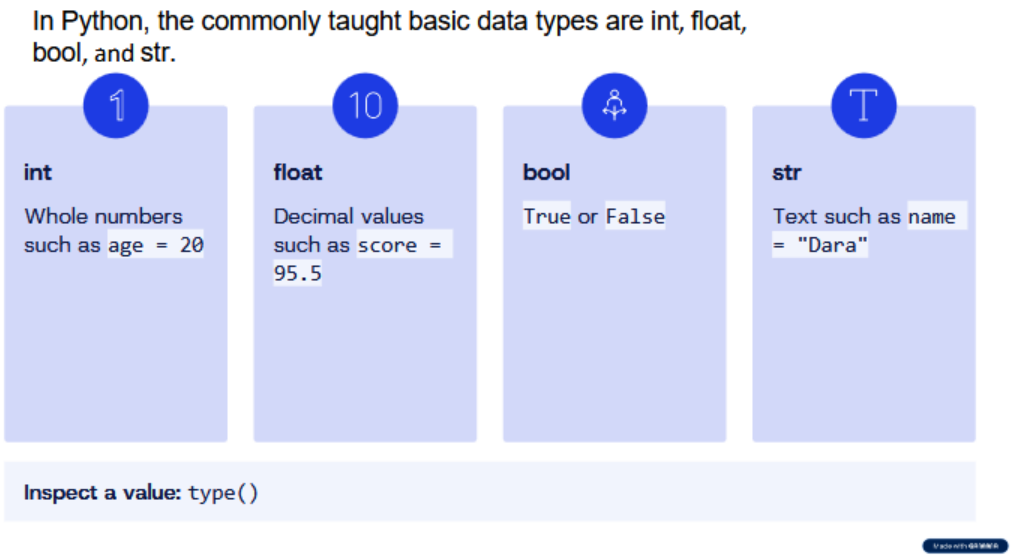
Name: CHHEAN SOVATHANA

ID: e20240331

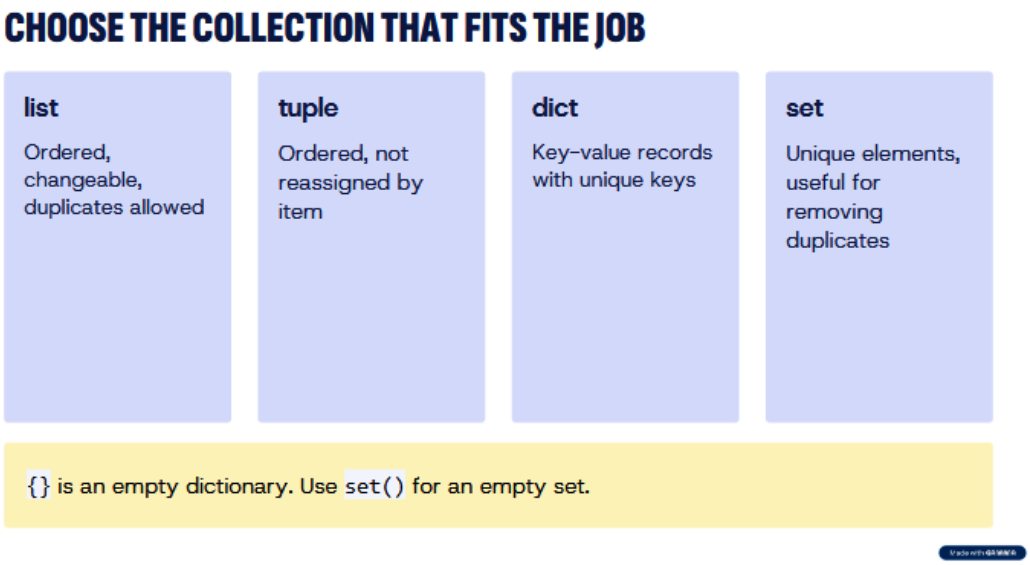
AMS-1A

I. Python Basic Data Types

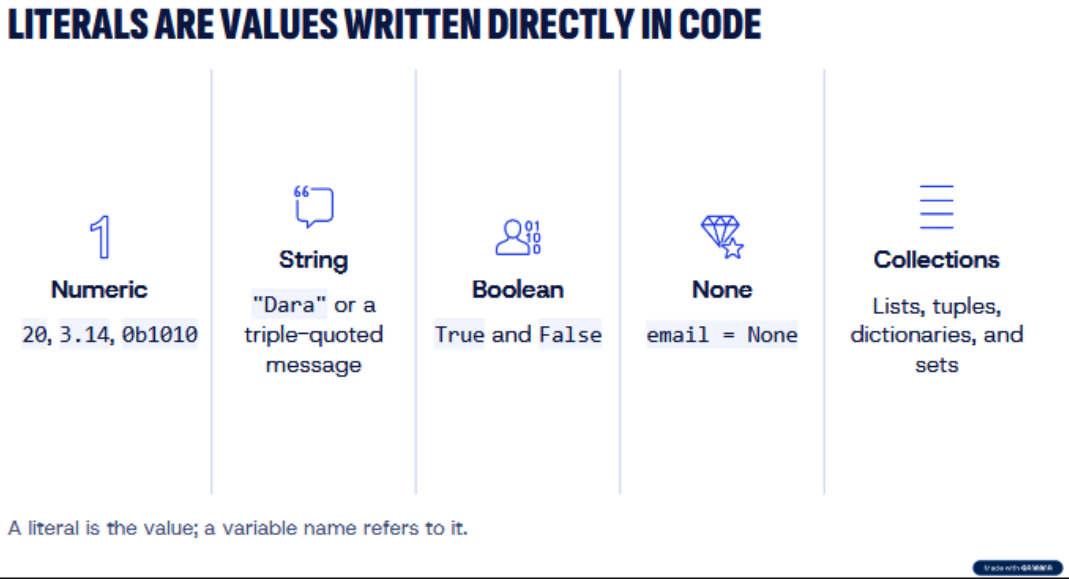
a) Primitive/Built-in Data Types (int, float, bool, str) is a basic type of data used to store a single value, such as a number, a decimal number, a true/false value, or text. In Python, the commonly taught basic data types are int, float, bool, and str.



b) Collection and Reference Types (list, tuple, dict, set, objects):



c) Literals in Python (numeric, string, Boolean, None, collection literals):



I. Practical example: storing student names, ages, scores, and student records.

example.py > ...

```python
students = {
    "01": {
        "name": "Dara",
        "age": 18,
        "score": 85
    },                          #list of students in set
    "02": {
        "name": "Sokha",
        "age": 19,
        "score": 90.2
    },
    "03": {
        "name": "Vanna",
        "age": 20,
        "score": 78
    }
}

for student_id, student in students.items():    # Display all records
    print("Student ID:", student_id)
    print("Name:", student["name"])     #str
    print("Age:", student["age"])       #int
    print("Score:", student["score"])   #float
    print("----------------")
```

```
Problems  Output  Debug Console  Terminal  Ports  Query Results  Filter
Student ID: 01
Name: Dara
Age: 18
Score: 85
----------------
Student ID: 02
Name: Sokha
Age: 19
Score: 90.2
----------------
Student ID: 03
Name: Vanna
Age: 20
```

II. Python Variables and Attributes

a) Local Variables are variables created inside a function or method. It exists only while that function runs.

b) Instance Attributes belong to one object. Every object keeps its own value.

c) Class Attributes (shared by all instances) defined in the class body. All instances share one value.

II. Practical example: a student class with individual names and a shared school name.

ex2.py > ...

```python
class Student:
    school_name = "Institute of Technology of Cambodia"  # Shared attribute

    def __init__(self, name, age, score):
        self.name = name      # Individual attribute
        self.age = age        # Individual attribute
        self.score = score    # Individual attribute

student1 = Student("Dara", 18, 85)  # Create student objects
student2 = Student("Sokha", 19, 90)
student3 = Student("Vanna", 20, 78)

print(student1.name, student1.age, student1.score, student1.school_name)    # Display student records
print(student2.name, student2.age, student2.score, student2.school_name)
print(student3.name, student3.age, student3.score, student3.school_name)
```

```
Problems  Output  Debug Console  Terminal  Ports  Query Results  Filter (e.g. text, !excludeText, t...  Code
[Running] python -u "c:\Users\LYHENG\OneDrive\Desktop\ITC_I3\TP_OOP\ex2.py"
Dara 18 85 Institute of Technology of Cambodia
Sokha 19 90 Institute of Technology of Cambodia
Vanna 20 78 Institute of Technology of Cambodia
```

III. Python Modifiers and Access Control

a) Public, Non-Public, and Private Naming Conventions:

Public, Non-Public and Private

Three naming styles tell other programmers how an attribute should be used.

```
class Account:
    def __init__(self):
        self.owner = "Sara" # public
        self._balance = 100 # non-public
        self.__pin = 1234 # private
```

name
Public: use it freely.

_name
Non-public: internal use, by convention.

__name
Private: Python renames it (mangling).

b) Access Control by Convention and Name Mangling:

Convention and Name Mangling

A single underscore is a hint. A double underscore makes Python rename the attribute.

```
acc = Account()
print(acc.owner) # Sara
print(acc._balance) # 100, works but avoid
print(acc.__pin) # AttributeError
print(acc._Account__pin) # 1234
```

_balance
Not enforced. It is a polite “please don’t touch”.

__pin
Stored as _Account__pin. This prevents accidents, it is not security.

c) The @staticmethod Decorator:

The @staticmethod Decorator

A function that lives inside the class but does not need the object or the class.

```
class MathTools:
    @staticmethod
    def add(a, b):
        return a + b

print(MathTools.add(2, 3)) # 5
```

No self, no cls

It receives only the arguments you pass.

Use it for

Helpers related to the class, like validation.

d) The @classmethod Decorator

The @classmethod Decorator

A method that receives the class itself as its first argument, called cls.

```
class Account:
    bank_name = "PyBank"

    @classmethod
    def set_bank(cls, name):
        cls.bank_name = name

Account.set_bank("NewBank")
print(Account.bank_name) # NewBank
```

cls

Gives access to class-level data.

Use it for

Changing shared settings or alternative constructors.

- Practical example: controlling access to a BankAccount class and defining class-level methods.

ex3.py > ...

```python
class BankAccount:
    bank_name = "ABA Bank"  # Class attribute

    def __init__(self, owner, balance):
        self.owner = owner                  # Public attribute
        self._account_type = "Saving"       # Protected by convention
        self.__balance = balance            # Private attribute

    def deposit(self, amount):              # Instance method
        if amount > 0:
            self.__balance += amount
            print("Deposit successful")
        else:
            print("Invalid amount")

    def get_balance(self):                  # Instance method
        return self.__balance

    @classmethod                            # Class method
    def change_bank_name(cls, new_name):
        cls.bank_name = new_name

    @staticmethod                           # Static method
    def is_valid_amount(amount):
        return amount > 0

account1 = BankAccount("Dara", 1000)        # Create an account
print("Owner:", account1.owner)             # Access public attribute
print("Bank:", BankAccount.bank_name)       # Access class attribute

print("Balance:", account1.get_balance())   # Access balance through a method

account1.deposit(500)                       # Deposit money
print("New balance:", account1.get_balance())

BankAccount.change_bank_name("National Bank")# Call a class method
print("New bank name:", BankAccount.bank_name)

print("Is amount valid?", BankAccount.is_valid_amount(200))# Call a static method
```

e) Abstract Classes and Methods (abc, @abstractmethod)

e. Abstract Classes: a blueprint, not a real thing

Like a job description

"Shape" says every shape must have an area, but not how to calculate it. Each subclass decides how.

Two rules

You cannot create an object from an abstract class. Every subclass must write all the abstract methods.

```
from abc import ABC,
abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

Shape()    # TypeError
```

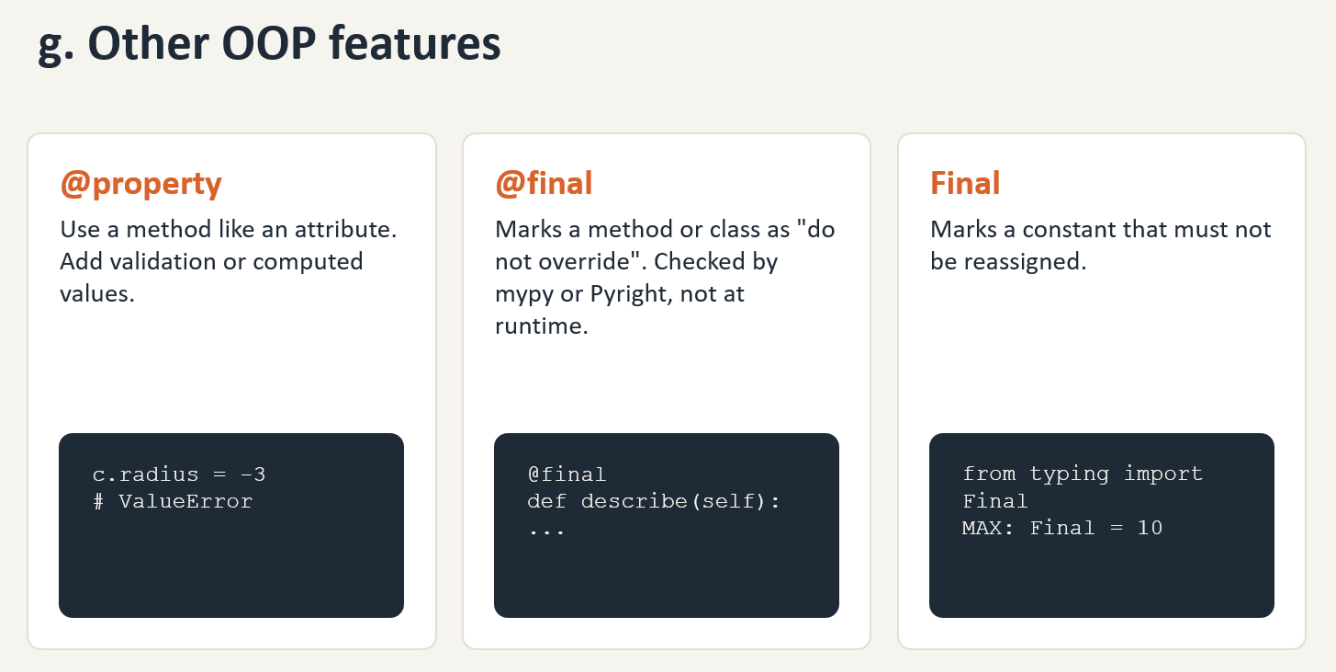
f) Access Control and Encapsulation

f. Access Control & Encapsulation: control who touches the data

Name	Level	What it means
radius	Public	Use freely
_radius	Protected	Internal, please don't touch (convention only)
__radius	Private	Renamed to _Circle__radius (name mangling)

Python hides by convention, not by force. It prevents accidents, not determined access.

g) Other OOP Features: @property, @final, and typing.final



➢ Practical example: an abstract Shape class with Circle and Rectangle subclasses.

ex4.py > Shape > describe

```python
from abc import ABC, abstractmethod
import math

class Shape(ABC):

    @property
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def describe(self):
        print("Area:", round(self.area, 2))
        print("Perimeter:", round(self.perimeter(), 2))

```

```python
class Circle(Shape):
    def __init__(self, radius):
        self.__radius = radius

    @property
    def area(self):
        return math.pi * self.__radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.__radius

class Rectangle(Shape):
    def __init__(self, width, height):
        self.__width = width
        self.__height = height

    @property
    def area(self):
        return self.__width * self.__height

    def perimeter(self):
        return 2 * (self.__width + self.__height)

circle = Circle(5)
rectangle = Rectangle(4, 6)

print("Circle:")
circle.describe()

print("\nRectangle:")
rectangle.describe()

```

Problems Output Debug Console Terminal Ports Query Results Filter (e.g. text, !excludeText, t...

```
[Running] python -u "c:\Users\LYHENG\OneDrive\Desktop\ITC_I3\TP_OOP\ex4.py"
Circle:
Area: 78.54
Perimeter: 31.42

Rectangle:
Area: 24
Perimeter: 20
```

IV. Python Operators

a) Arithmetic Operators (+, -, *, /, //, %, **): Arithmetic operators are used to perform mathematical calculations. + is addition, - is subtraction, * is multiplication, / is division, // is floor division, % returns the remainder, and ** is exponentiation.

b) Comparison Operators (==, !=, >, <, >=, <=): Comparison operators compare two values and return True or False. == means equal to, != means not equal to, > means greater than, < means less than, >= means greater than or equal to, and <= means less than or equal to.

c) Bitwise Operators (&, |, ^, ~, <<, >>): Bitwise operators perform operations on the binary bits of integers. & is AND, which returns 1 when both bits are 1; | is OR, which returns 1 when at least one bit is 1; ^ is XOR, which returns 1 when the bits differ; ~ is NOT, which inverts the bits; << shifts bits to the left; and >> shifts bits to the right.

d) Logical Operators (and, or, not): Logical operators combine conditions or reverse their truth values. and returns True if both conditions are true, or returns True if at least one condition is true, and not reverses the result.

➢ Practical example: calculating student grades and checking eligibility.

ex5.py > students

```python
students = [
    {"name": "Dara", "scores": [90, 85, 95]},
    {"name": "Sokha", "scores": [60, 55, 65]}
]

for student in students:
    average = sum(student["scores"]) / 3

    if average >= 90:
        grade = "A"
    elif average >= 80:
        grade = "B"
    elif average >= 70:
        grade = "C"
    elif average >= 60:
        grade = "D"
    else:
        grade = "F"

    print("Name:", student["name"])
    print("Average:", average)
    print("Grade:", grade)

    if average >= 50:
        print("Status: Passed")
    else:
        print("Status: Failed")

    if average >= 85:
        print("Scholarship: Eligible")
    else:
        print("Scholarship: Not eligible")

    print("----------------")
```

e) Assignment Operators (=, +=, -=, *=, etc.): Assignment operators are used to assign values to variables or update their values. = assigns a value, += adds and assigns, -= subtracts and assigns, *= multiplies and assigns, /= divides and assigns, //= performs floor division and assigns, %= calculates the remainder and assigns, and **= raises a value to a power and assigns.

f) Special Operators: Identity (is, is not), Membership (in, not in), and Conditional Expressions (x if condition else y): Special operators perform specific operations in Python. Identity operators (is, is not) check whether two variables refer to the same object. Membership operators (in, not in) check whether a value exists in a sequence or collection. Conditional expressions (x if condition else y) return one value if the condition is true and another if it is false.

g) Operator Precedence and Associativity: Operator precedence determines the order in which operators are evaluated in an expression, while associativity determines the evaluation order when operators have the same precedence. Parentheses () can be used to control the order of evaluation.

➢ Practical example: updating object attributes, checking membership in collections, and evaluating expressions.

ex6.py > Student

```python
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

student1 = Student("Dara", 80)

# Assignment operators
student1.score += 10
print("Updated score:", student1.score)

# Membership operators
students = ["Dara", "Sokha", "Vanna"]
print("Dara" in students)
print("Malis" not in students)

# Identity operator
student2 = student1
print(student1 is student2)

# Conditional expression
result = "Pass" if student1.score >= 50 else "Fail"
print("Result:", result)

# Operator precedence
total = 10 + 5 * 2
print("Total:", total)
```

```
Problems  Output  Debug Console  Terminal  Ports  Query Results  Filter (e.g. text, !excludeText, t...
[Running] python -u "c:\Users\LYHENG\OneDrive\Desktop\ITC_I3\TP_OOP\ex6.py"
Updated score: 90
True
True
True
Result: Pass
Total: 20
```

V. Python Decision-Making Statements

a) The if Statement

The if Statement

Runs a block of code **only when** a condition is True. Otherwise it does nothing.

```
score = 75
if score >= 60:
    print("You passed!")
```

```
Output   You passed!
```

b) The if-else Statement

The if-else Statement

Chooses between **exactly two** paths.

If the condition is True, the **if** block runs. If not, the **else** block runs.

```
score = 45
if score >= 60:
    print("Pass")
else:
    print("Fail")
```

```
Output   Fail
```

c) The if-elif-else Statement

The if-elif-else Statement

We use this statement when we have more than two conditions.

Python checks each condition from top to bottom.

```
mark = 83
if mark >= 90:
    print("Excellent")
elif mark >= 70:
    print("Good")
else:
    print("Keep going")
```

```
Output  Good
```

Practical example: determining student pass/fail status and assigning letter grades.

As the same example as IV. Python Oparators

ex5.py > students

```
students = [
    {"name": "Dara", "scores": [90, 85, 95]},
    {"name": "Sokha", "scores": [60, 55, 65]}
]

for student in students:
    average = sum(student["scores"]) / 3

    if average >= 90:
        grade = "A"
    elif average >= 80:
        grade = "B"
    elif average >= 70:
        grade = "C"
    elif average >= 60:
        grade = "D"
    else:
        grade = "F"

    print("Name:", student["name"])
    print("Average:", average)
    print("Grade:", grade)

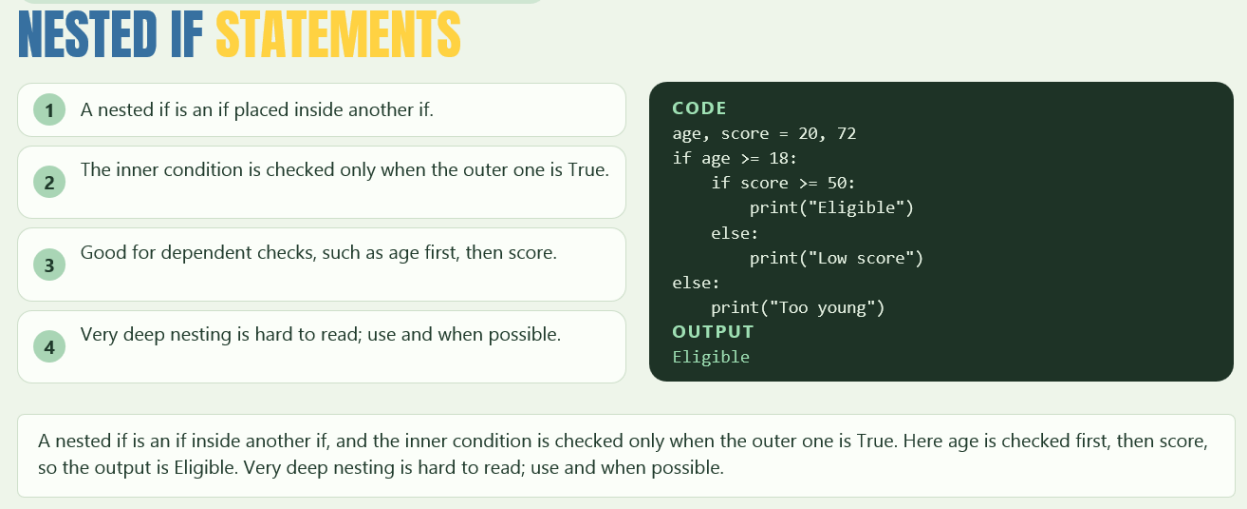
    if average >= 50:
        print("Status: Passed")
    else:
        print("Status: Failed")

    if average >= 85:
        print("Scholarship: Eligible")
    else:
        print("Scholarship: Not eligible")

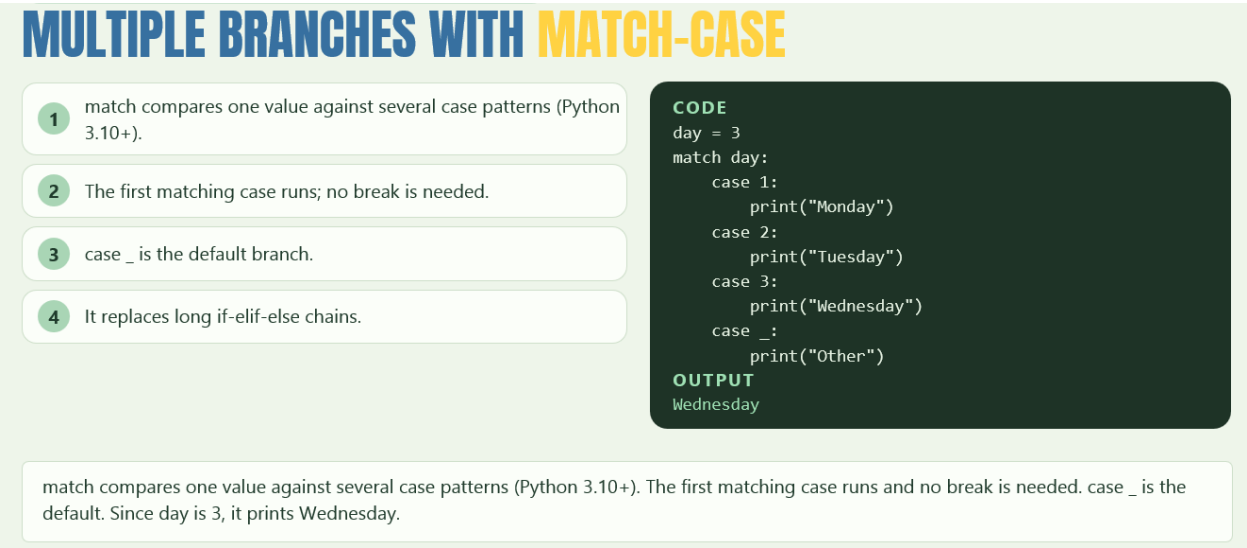
    print("----------------")
```

```
Problems  Output  Debug Console  Terminal  Ports  Query Results  Filter (e.g. text, !excludeText, t..
[Running] python -u "c:\Users\LYHENG\OneDrive\Desktop\ITC_I3\TP_OOP\ex5&7.py"
Name: Dara
Average: 90.0
Grade: A
Status: Passed
Scholarship: Eligible
---------------
Name: Sokha
Average: 60.0
Grade: D
Status: Passed
Scholarship: Not eligible
---------------
```

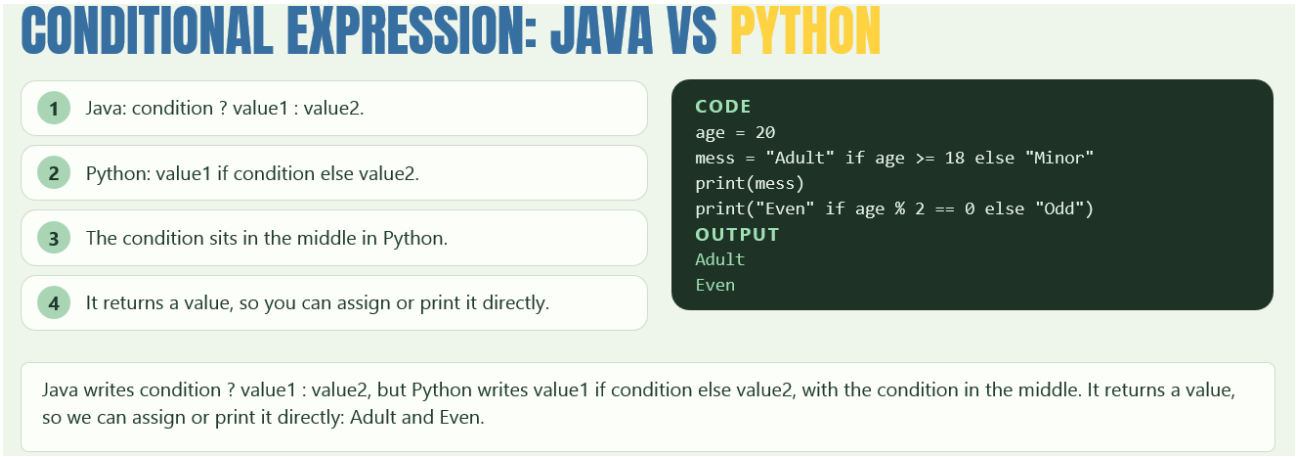
d) Nested if Statements:



e) Multiple-Branch Selection Using match-case (Python 3.10+) or if-elif-else:



f) Conditional Expressions (condition ? value1 : value2 in Java becomes value1 if condition else value2 in Python):



➢ Practical example: a menu-driven student management system.

ex8.py > ...

```python
students = []

while True:
    print("\nChoose a choice: 1.Add 2.Show 3.Exit")
    choice = int(input("Choose: "))

    match choice:
        case 1:
            name = input("Name: ")
            if name != "":
                students.append(name)
                print("Student added successfully!")
            else:
                print("Name cannot be empty!")

        case 2:
            print(students)

        case 3:
            print("Exit program")
            break

        case _:
            print("Invalid choice!")
```

```
OUTPUT
1.Choose a choice: 1.Add 2.Show 3.Exit
Choose: 1
Name: Panha
1.Choose a choice: 1.Add 2.Show 3.Exit
Choose: 2
['Panha']
1.Choose a choice: 1.Add 2.Show 3.Exit
Choose: 3
Exit program
```

VI. Python Loops and Loop Control

a) The while Loop: repeats a block of code as long as the condition is true.
b) The for Loop: Repeats a block of code for each item in a sequence or a specified number of times.
c) Repetition with for and while (Python has no native do-while loop): Python uses for and while loops for repetition, and while True with break can imitate a do-while loop.
d) Loop Control Statements: break, continue, and pass: break stops the loop, continue skips to the next iteration, and pass does nothing as a placeholder.
e) The break Statement: Immediately terminates the loop and continues executing the code after it.

➢ Practical example: iterating through student objects, calculating averages and searching for a student.

ex9.py > ...

```python
class Student:
    def __init__(self, name, grades):
        self.grades = grades

students = [Student("Ana", [90, 80]),
            Student("Ben", [70, 60])]

for s in students:
    avg = sum(s.grades) / len(s.grades)
    print(avg)  # 85.0, 65.0
for s in students:
    if s.name == "Ben":
        print("Found:", s.name)
        break
else:
    print("Not found")

```

Problems Output Debug Console Terminal Ports Query Results Filter (e.g. text,

```
[Running] python -u "c:\Users\LYHENG\OneDrive\Desktop\ITC_I3\TP_OOP\ex9.py"
85.0
65.0
Found: Ben
```

- Important differences between Java and Python:

Topic	Java	Python
Blocks	`Braces { }`	`Indentation`
Counting loop	`for (int i = 0; i < n; i++)`	`for i in range(n)`
Each element	`for (Student s : list)`	`for s in students`
do-while	`Exists`	`None: while True + break`